

Diffusion Models

Ulf Krumnack

Robin Rawiel

*“Time’s arrow dictates that structure eventually yields to chaos,
but intelligence is the art of reversing the flow.”*

1 The Arrow of Time: Physical Intuition of Diffusion

In the natural world, systems inherently tend to move from order to disorder. If you carefully place a single drop of dark ink into a clear glass of water, you will initially see a highly structured, dense cloud of color. However, as the seconds pass, the water molecules constantly collide with the ink particles in a process of random thermal motion. Slowly but inevitably, the ink spreads out, blurring its sharp edges until the entire glass becomes a uniform, featureless shade of pale gray. In physics, this process is known as diffusion, and it represents a continuous increase in entropy, or disorder.

This physical process is straightforward to observe forward in time, but it is practically impossible to reverse. You cannot stir a glass of gray water and expect the ink to spontaneously gather itself back into a perfect, spherical drop. The [arrow of time](#) points strictly from structure to chaos (see Figure 1). While we can easily describe how the ink diffuses forward, finding a way to accurately reverse this sequence of events in the physical world is naturally beyond our reach.

[arrow of time](#)

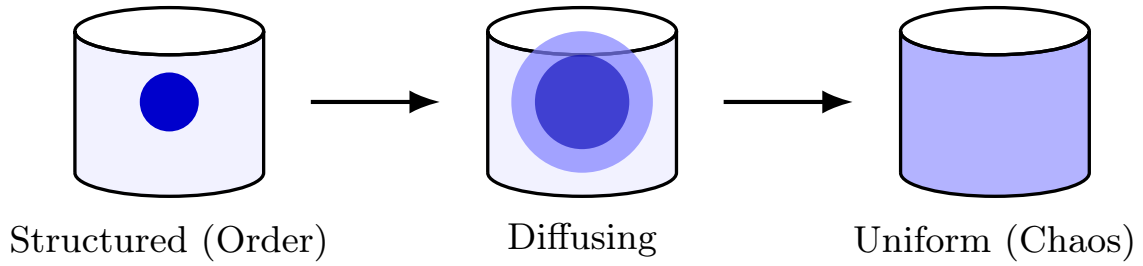
In deep learning, we can use this physical metaphor to design a highly effective family of [generative models](#) known as [diffusion models](#). We can think of a crisp, highly structured photograph from our training data as our initial drop of ink. If we mathematically add a small amount of random noise to this image over and over again, we are simulating a [forward diffusion process](#). Eventually, the original image will be completely degraded, turning into a screen of pure, chaotic static. This step is entirely prescribed and requires no machine learning; we are simply destroying the information in the image step by step.

[diffusion models](#)

[forward diffusion process](#)

The central idea of diffusion models is that while physical laws do not allow us to reverse time, machine learning provides a way to estimate the reverse process mathematically. If we can train a neural network to carefully observe the destruction process, it can learn to reverse the steps. By learning exactly how structure breaks down into noise, the network can also learn how to extract structure out of noise. Generating a new image then becomes the process of starting with a canvas of pure random noise and systematically “un-diffusing” it. By applying the trained network iteratively, we gradually remove the noise step by step until a meaningful picture emerges.

Physical Process: Ink Diffusing



Mathematical Process: Adding Noise

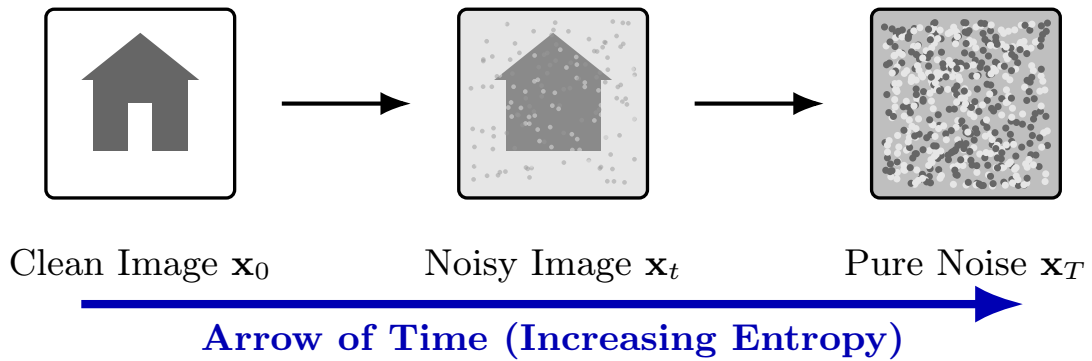


Figure 1: The forward diffusion process. Just as a highly structured drop of ink naturally dissipates into a uniform cloud within a glass of water, a structured image can be systematically degraded into pure, featureless noise. In both cases, the natural arrow of time flows from order to chaos.

2 Formalizing the Forward Diffusion Process

Let us translate the physical metaphor of ink diffusing in water into a mathematical framework. We begin with a clean data point from our training set, such as a sharp photograph, which we denote as $\mathbf{x}_0$. We then apply a sequence of noise-adding steps over a total of T steps. The intermediate, increasingly noisy versions of our image will be denoted as $\mathbf{x}_1$, $\mathbf{x}_2$, and so on, all the way up to $\mathbf{x}_T$, which will be effectively pure random noise.

To formalize this step-by-step destruction, we use a concept from probability theory known as a [Markov process](#). The defining characteristic of a Markov process is its “memoryless” property. When computing the state of a system at a given step t , the only information we need is the immediate previous state at step $t - 1$. We do not need to know the entire history of how the system reached that previous state.

Markov
process

Math: Markov Process

In mathematical terms, the probability of the current state depends only on the previous one:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \dots, \mathbf{x}_0) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

This property fits our [forward diffusion process](#) perfectly. To create the noisy image at step t , we simply take the image from step $t - 1$ and add a little bit of random variation to it. We do not need to look at the original image $\mathbf{x}_0$ to perform this single step.

At each step, we carefully control how much noise is added using a parameter β_t . The sequence of these parameters over all T steps is called the [noise schedule](#). A common choice is to start with a very small β and gradually increase it, so the image degrades slowly at first and faster later on. The mathematical operation to compute the next state is given by:

noise
schedule

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon$$

Here, ϵ is random noise sampled from a standard normal distribution, meaning each pixel gets a small random push. Notice that we multiply the previous state $\mathbf{x}_{t-1}$ by $\sqrt{1 - \beta_t}$. This factor slightly shrinks the original pixel values. If we only added noise without shrinking the original image, the values in our image would grow larger and larger over time. By balancing the shrinking factor and the noise factor, we keep the overall variance of the image roughly constant throughout the process. In terms of probability distributions, we write this single transition as a normal distribution:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

While this step-by-step formula gives us a clear path from data to noise, it introduces a practical problem. Imagine we are training our neural network and we need to simulate the noisy image at step $t = 500$. Using the formula above, we would have to compute step 1, then step 2, all the way to 500 in sequence. This iterative computation would make training very slow.

Fortunately, there is a mathematical shortcut that allows us to compute the noise at an arbitrary time t in a single step (see Figure 2). This shortcut relies on the fact that when you

add multiple normal distributions together, the result is simply another normal distribution. To use this shortcut, we introduce a new variable $\alpha_t = 1 - \beta_t$. Furthermore, we define $\bar{\alpha}_t$ (read as “alpha bar”) as the cumulative product of all alphas up to time t :

$$\bar{\alpha}_t = \alpha_1 \cdot \alpha_2 \cdots \alpha_t$$

Because α_t represents the amount of the original signal that is kept at each step, $\bar{\alpha}_t$ elegantly represents the total fraction of the original image $\mathbf{x}_0$ that survives after t steps. By chaining the individual steps together and combining the noise terms mathematically, we can write the state $\mathbf{x}_t$ directly in terms of the starting image $\mathbf{x}_0$:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$

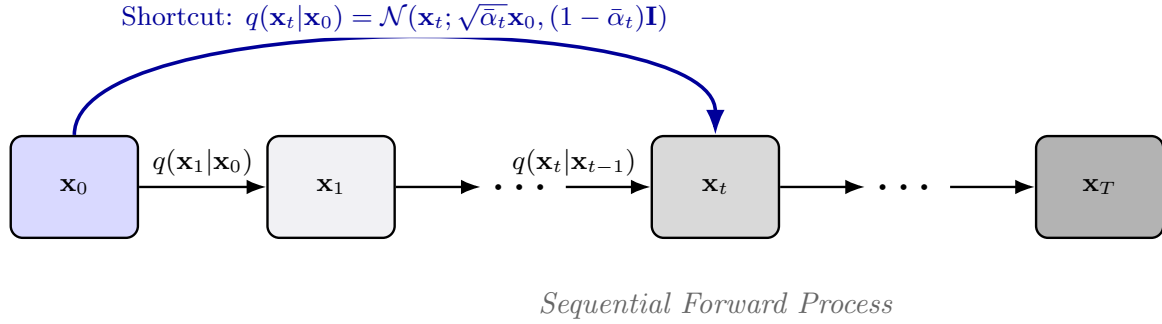


Figure 2: The Markov chain and the mathematical shortcut. We can either simulate the step-by-step degradation of the image through multiple sequential steps, or we can use the cumulative factor $\bar{\alpha}_t$ to jump directly from the pristine data $\mathbf{x}_0$ to any noisy state $\mathbf{x}_t$ in a single, efficient operation.

In this final equation, $\boldsymbol{\epsilon}$ is again a single sample of standard normal noise, but it now accounts for all the tiny noise additions from the intermediate steps combined. This formula is highly practical. It gives us the ability to pick any arbitrary time step t , immediately blend the original image with the correct proportion of noise, and supply the result to our neural network for training, completely bypassing the need to simulate the intermediate steps.

3 Reversing the Arrow of Time

Now that we have a mathematical recipe for turning any training image into pure noise, we arrive at the core challenge of diffusion models: we need to run time backwards. In the forward direction, our process takes a structured image $\mathbf{x}_0$ and produces a noisy image $\mathbf{x}_t$. This is governed by a known probability distribution $q(\mathbf{x}_t | \mathbf{x}_{t-1})$. For the [reverse diffusion process](#), we want to find the opposite transition: given a noisy image at step t , we want to predict the slightly cleaner image at step $t - 1$. We denote this reverse probability distribution as $p(\mathbf{x}_{t-1} | \mathbf{x}_t)$.

reverse
diffusion
process

Finding this reverse distribution using traditional statistics is practically impossible. To accurately guess the slightly cleaner image, you would technically need to know the entire

distribution of all possible images in the world, which is exactly the highly complex data distribution we are trying to learn in the first place. However, physics provides us with a profound insight. If the amount of noise added in each forward step β_t is extremely small, the reverse step will have the exact same functional form as the forward step. Since we used a normal distribution to add noise, the reverse step is also guaranteed to be a normal distribution.

This insight simplifies our task. We do not need to model an impossibly complex distribution from scratch; we only need to find the mean and variance of a normal distribution. Let us formalize this reverse step. We will use a neural network with parameters θ to estimate this normal distribution, which we write as $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$. The network’s job is to output the predicted mean $\mu_\theta(\mathbf{x}_t, t)$ and a predicted variance $\Sigma_\theta(\mathbf{x}_t, t)$. In practice, it has been found that the variance can often be fixed to the known constants from the noise schedule, meaning the neural network only has to focus on predicting the mean. The problem of reversing time thus boils down to training a neural network to look at a noisy image $\mathbf{x}_t$ and predict the mean of the slightly cleaner image $\mathbf{x}_{t-1}$.

4 Predicting the Noise

While predicting the mean of the previous image is a valid approach, modern diffusion models take a slightly different, more robust path. Through mathematical reformulations, it turns out that predicting the mean of the reverse step is completely equivalent to predicting the exact noise ϵ that was added to the image. This subtle shift in perspective makes the learning task highly intuitive: we give the network a noisy image, and we ask it to point out the noise so we can subtract it.

We define our neural network model as $\epsilon_\theta(\mathbf{x}_t, t)$. This model takes two inputs: the noisy image $\mathbf{x}_t$ and the current time step t . Its output is an estimate of the noise tensor, which has the exact same dimensions as the image itself.

To train this network, we take great advantage of the $\bar{\alpha}_t$ mathematical shortcut we established earlier. Recall that this shortcut allows us to jump instantly to any time step t by blending the original image $\mathbf{x}_0$ with a single sample of pure noise ϵ . Because we generated this noisy image ourselves, we know the exact “ground truth” noise ϵ that is hidden inside it. We can simply ask our neural network ϵ_θ to examine the noisy image and predict the noise. We then compare the network’s prediction against the true noise using a standard [squared error loss](#).

This results in a elegant training objective. We randomly sample a clean image $\mathbf{x}_0$ from our training data, we randomly pick a time step t , and we sample random noise ϵ . We then mix the image and the noise using our $\bar{\alpha}_t$ formula to create $\mathbf{x}_t$. Finally, we compute our loss function:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_0, \epsilon, t} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

This formula reads much more complex than its implementation. It simply states that we want to minimize the Euclidean distance between the actual noise ϵ and the noise our model

predicts. This specific mathematical framework, training a neural network to systematically denoise an image by predicting the added noise across various time steps, is formally known as a [Denoising Diffusion Probabilistic Model](#), commonly abbreviated as [DDPM](#). By repeatedly minimizing this [empirical loss](#) over thousands of batches, the DDPM becomes highly skilled at identifying and isolating noise patterns at any stage of the diffusion process.

[Denoising
Diffusion
Probabilistic
Model](#)

5 The U-Net Architecture for Diffusion

To perform this noise prediction task, we need a neural network architecture that can take an image as input and produce an output of the exact same spatial dimensions. The [U-Net](#), which we encountered earlier in the context of image segmentation and autoencoders, is the perfect tool for this job.

When a noisy image is fed into the U-Net, it first passes through the [encoder](#) pathway. This pathway consists of convolutional layers and pooling operations that progressively reduce the spatial width and height of the image while increasing the number of feature channels. This allows the network to understand the global context of the image. For instance, the network needs to recognize whether it is looking at the rough shape of a dog, a car, or a landscape, even through heavy static.

Once the image is compressed into a compact representation at the bottleneck, it passes through the [decoder](#) pathway. Here, upsampling operations gradually restore the image to its original resolution. The crucial components that make the U-Net so effective are the [skip connections](#). These connections bypass the bottleneck, carrying detailed, high-resolution spatial information directly from the encoder layers to the corresponding decoder layers. This ensures that the final output aligns perfectly with the pixels of the input image, which is essential when the goal is to pinpoint exact pixel-wise noise patterns.

There is one final, critical modification we must make to the standard U-Net. Our network ϵ_θ needs to know the time step t . The noise at step $t = 10$ is barely visible, mostly fine-grained static, while the noise at step $t = 900$ is overwhelming, obliterating almost all structure. The network must use different strategies depending on how degraded the image is. We achieve this by encoding the scalar integer t into a vector, similar to the embeddings used in natural language processing, and injecting this time signal into the various layers of the U-Net. This allows the U-Net to adapt its feature extraction process based on the current stage of the diffusion timeline.

6 Training the Diffusion Model

With our objective and our U-Net architecture established, we can now assemble the complete training procedure. The training loop is surprisingly straightforward because the forward process requires no sequential steps. We can train the model on random time steps in parallel, making the optimization highly efficient using standard [batch gradient descent](#) techniques like the Adam optimizer.

During each training iteration, we draw a [mini-batch](#) of clean images from our dataset. For every single image in the batch, we independently sample a random time step t from a uniform distribution between 1 and T . We also draw a corresponding batch of random noise vectors from a standard normal distribution. By applying our single-step $\bar{\alpha}_t$ formula, we instantly corrupt our clean images to their respective random time steps. We then feed these corrupted images, along with their time steps, into the U-Net. We calculate the squared error between the U-Net's predictions and the actual noise we added, and we use [backpropagation](#) to update the model parameters θ .

We can summarize this training loop cleanly using Python-like pseudocode. Notice how the mathematical equations translate directly into simple, vectorized programming operations:

```
def train_step(clean_images, max_time_steps, model, optimizer):
    # Determine how many images are in the current batch
    batch_size = clean_images.shape[0]

    # 1. Sample a random time step for each image in the batch
    t = random_integer(low=1, high=max_time_steps, size=batch_size)

    # 2. Sample pure random noise for each image
    true_noise = random_normal(size=clean_images.shape)

    # 3. Look up the cumulative alpha values for the sampled time steps
    alpha_bar = get_alpha_bar_schedule(t)

    # 4. Mix clean images and noise using the single-step shortcut
    noisy_images = sqrt(alpha_bar) * clean_images + \
        sqrt(1 - alpha_bar) * true_noise

    # 5. Pass noisy images and time steps to the U-Net
    predicted_noise = model(noisy_images, t)

    # 6. Compute the squared error loss
    loss = mean_squared_error(true_noise, predicted_noise)

    # 7. Update model parameters via gradient descent
    loss.backward()
    optimizer.step()

    return loss
```

By repeating this process over many [epochs](#), the network learns an internal representation of how natural images are constructed and degraded, preparing it for the generation phase.

7 Generating New Images

Once the model is fully trained, we can finally use it as a **generative model** to create entirely new images from scratch. While training was fast thanks to the mathematical shortcut that allowed us to jump to any time step, generation must strictly follow the iterative, step-by-step process. We must simulate the full reverse Markov chain, starting from absolute chaos and systematically guiding it towards order.

We can visualize this generation procedure as a sequence flowing backwards in time, from right to left (see Figure 3). We begin on the far right by sampling an image of pure random noise. Then, we repeatedly apply a noise removal step, moving leftward through time, until we finally arrive at a clean image that belongs to our data distribution.

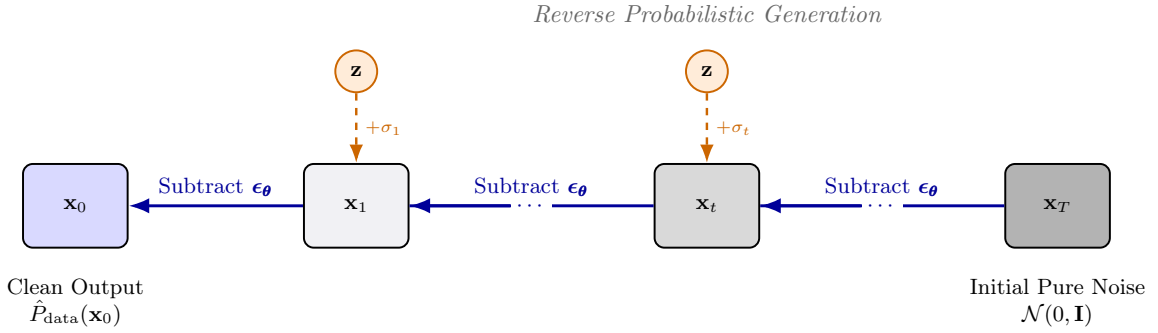


Figure 3: The generative reverse process. Starting from pure random noise on the right, the model iteratively estimates and subtracts noise, while continuously injecting fresh random noise (Langevin dynamics). This process steadily guides the chaotic signal back to a sharp, realistic sample on the left.

We can express this general idea, along with a simplified formula for the noise removal step, as follows:

$$\underbrace{\mathbf{x}_0 \sim \hat{P}_{\text{data}}(\mathbf{x})}_{\text{model output}} \leftarrow \underbrace{\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right)}_{\text{deterministic noise removal (repeat } T \text{ times)}} \leftarrow \underbrace{\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})}_{\text{initial pure noise}}$$

Let us take a moment to unpack the two specific scaling factors present in this deterministic noise removal formula. First, consider the outer multiplier $\frac{1}{\sqrt{\alpha_t}}$. Recall that during the forward diffusion process, we multiplied the image by $\sqrt{\alpha_t}$ at each step to slightly shrink the pixel values, ensuring the overall variance did not explode when we added noise. Because we are now reversing the process, we must divide by this exact same factor to restore the image to its proper scale.

Second, consider the fraction $\frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}}$ located in front of our network's prediction $\boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)$. Our neural network was trained to predict the total, cumulative noise that was added to the original image from the very beginning up to time t . However, we do not want to subtract all of this noise at once; we only want to take a single step backward in time. This fraction

serves as the correct proportional weight to scale the total predicted noise down to the precise amount of incremental noise that belongs only to the current step t .

At first glance, this purely deterministic approach seems perfectly logical: if the network accurately scales and subtracts the noise, we should simply reveal the cleaner image. However, taking this pure approach in practice leads to a significant problem. A neural network is never perfectly accurate. When it looks at a heavily degraded image, there are many possible original images that could have resulted in that specific noisy pattern. Because the network was trained using a [squared error loss](#), its optimal mathematical strategy when uncertain is to predict the statistical average of all those possible valid images.

If we strictly follow the deterministic formula, we end up stacking these “average” predictions on top of each other. In pixel space, the average of many possible sharp edges is a soft, blurry gradient. Consequently, this deterministic reverse process consistently outputs images that are extremely smooth, blurry, and devoid of realistic textures and fine details. It effectively collapses the rich diversity of the [manifold hypothesis](#) space into a safe, featureless middle ground.

To solve this problem, we must acknowledge that the original forward diffusion process was not deterministic; it was a probabilistic Markov chain heavily reliant on randomness. To accurately reverse it, our generation process must also be probabilistic, which is exactly why this foundational framework carries the name Denoising Diffusion *Probabilistic* Models. We achieve this probabilistic reversal by intentionally injecting a fresh sample of random noise back into the image at every single step. We can understand this through a physical concept known as [Langevin dynamics](#). In physics, Langevin dynamics describes the movement of particles in a fluid. A particle experiences a systematic, deterministic force pushing it toward a state of lower energy, much like gravity pulling a ball down a hill. At the same time, the particle is constantly bombarded by surrounding fluid molecules, resulting in random thermal jittering.

[Langevin
dynamics](#)

In the context of diffusion models, our deterministic noise removal formula acts as the systematic force, guiding the noisy image toward regions of high probability where realistic images live. The fresh random noise we inject acts as the thermal jittering. This random jostling is absolutely crucial because it prevents the generation process from getting stuck in those safe, blurry, “average” states. It continuously shakes the image, forcing it to actively explore the space of possibilities until it settles into a sharp, highly detailed, and specific configuration.

This introduces the complete formula for the reverse step, which includes the additional noise term $\mathbf{z}$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

Here, $\mathbf{z}$ is a new sample of standard normal noise, and σ_t is a variance scaling factor. The choice of σ_t is not arbitrary; it must carefully match the uncertainty of the reverse step. In practice, σ_t is typically derived directly from the forward noise schedule, commonly set to the standard deviation of the forward step, which is $\sqrt{1 - \alpha_t}$, or a closely related variance

value. Crucially, as we step backward in time and t approaches zero, the value of σ_t gets smaller and smaller. This acts like a cooling process in physics: as the image approaches its final form, the thermal jittering subsides, allowing the model to lock in the final, crisp details without leaving any residual static in the end result. Naturally, this injection of noise happens at every step except the very last one, ensuring the final output $\mathbf{x}_0$ is entirely clean.

Here is the algorithmic view of this complete generation process:

```
def generate_image(model, max_time_steps, image_shape):
    # 1. Start with pure random noise (the right side of our diagram)
    x = random_normal(size=image_shape)

    # 2. Loop backwards through time
    for t in reversed(range(1, max_time_steps + 1)):
        # Predict the noise using the trained U-Net
        predicted_noise = model(x, t)

        # Look up the schedule constants for time t
        alpha = get_alpha_schedule(t)
        alpha_bar = get_alpha_bar_schedule(t)
        sigma = get_sigma_schedule(t)

        # Calculate the deterministic part of the reverse step
        x = (1 / sqrt(alpha)) * \
            (x - ((1 - alpha) / sqrt(1 - alpha_bar)) * predicted_noise)

        # Add random noise to maintain sharpness
        # (except for the very last step t=1)
        if t > 1:
            z = random_normal(size=image_shape)
            x = x + sigma * z

    # The final x is our cleanly generated image
    return x
```

Watching this process unfold visually is highly instructive. The initial steps refine the pure noise into broad, cloudy color blobs. Halfway through the process, the general shapes and outlines of objects begin to confidently emerge from the fog. In the final steps, guided by the constant balancing act of deterministic structure building and stochastic Langevin dynamics, the model resolves the finest high-frequency details. It ultimately outputs a crisp, coherent, and entirely novel image that elegantly mimics the underlying distribution of the training data.

8 The Probability Landscape and Continuous Flows

To deeply understand how diffusion models navigate the complex journey between noise and realistic data, it is helpful to construct a mental image. In the first part of this course, we explored the concept of a [loss landscape](#), a multidimensional terrain where valleys represented states of low error. Our goal during training was to find the steepest downhill direction to minimize our objective. Now, we can imagine a related but conceptually distinct terrain: a probability landscape. In this landscape, the peaks of high mountains represent highly structured, realistic images from our training data. The vast, flat valleys and plains surrounding these mountains represent pure, chaotic random noise.

If we drop a degraded, noisy image somewhere on the slopes of this terrain, we can mathematically ask which direction is the steepest way uphill toward a realistic image. This uphill vector field is called the [score function](#), written mathematically as $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$. Just as the [gradient](#) in a loss landscape points toward higher error, the score function points toward higher probability. Finding a way to map out this uphill direction is known as [score matching](#) (see Figure 4).

[score
function](#)

[score
matching](#)

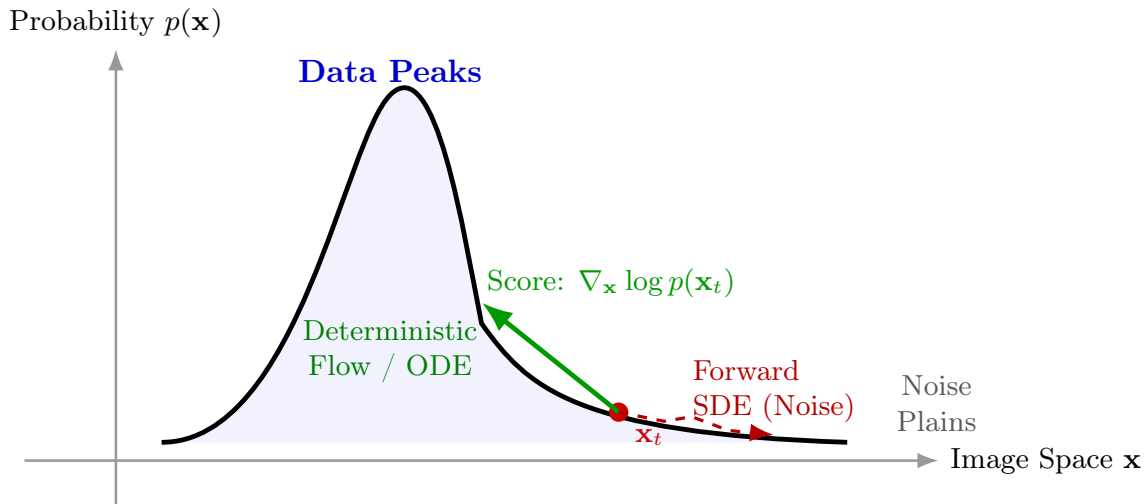


Figure 4: The probability landscape. Realistic data lives at the high peaks, while pure noise spans the flat valleys. The stochastic forward process pushes the image downhill via an erratic path. Conversely, the score function provides a vector field pointing directly uphill, allowing the model to navigate back to the data distribution.

With this landscape in mind, we can visualize the forward diffusion process. When we systematically add random noise to a clean image, we are actively pushing it off a mountain peak and sending it tumbling down into the flat plains of pure noise. Because we add random noise at every discrete step, this descent is not a straight, deterministic slide. Instead, it is a bumpy, erratic path.

This brings us to a conceptual leap. So far, we have treated the time t as a sequence of distinct, integer steps. However, in the natural world, processes like ink diffusing in water do not happen in sudden jumps; they happen as a smooth, continuous flow. We can establish

an interesting connection here to our iterative optimization techniques like [gradient descent](#). In gradient descent, we calculate a continuous mathematical gradient, but we use it to take discrete jumps governed by a [learning rate](#). With diffusion models, we can reverse this logic: we started by defining discrete random jumps, but we can mathematically smooth them out into a continuous flow.

These continuous randomized flows can be mathematically modeled using a framework called a [Stochastic Differential Equation](#), or [SDE](#). Instead of discrete steps, an SDE describes how the state of our image $\mathbf{x}_t$ changes over a microscopic time interval dt . The forward diffusion process can be elegantly written as an SDE:

[Stochastic
Differential
Equation](#)

$$d\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t)dt + g(t)d\mathbf{w}$$

This equation breaks the continuous velocity of our pixels into two distinct components. The first term, $\mathbf{f}(\mathbf{x}_t, t)$, is the [drift coefficient](#). It is a purely deterministic function that systematically scales the pixel values, acting as a steady, predictable current. The second term introduces the stochasticity: $g(t)$ is the [diffusion coefficient](#) controlling the amount of noise, and $d\mathbf{w}$ represents the [Wiener process](#). You can intuitively think of the Wiener process as the mathematical description of continuous random jitter, much like the erratic, microscopic bouncing of a pollen grain suspended in water. At every infinitesimal fraction of a second, $d\mathbf{w}$ injects a tiny, unpredictable push.

[drift
coefficient](#)
[diffusion
coefficient](#)
[Wiener
process](#)

By framing our model as a continuous SDE, we unlock a rigorous way to run time backward. According to the mathematics of SDEs, the reverse process, flowing continuously from pure noise at T back to a pristine image at 0, follows an equation with a strikingly similar structure:

$$d\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - g(t)^2 \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \right] dt + g(t)d\bar{\mathbf{w}}$$

Notice how the deterministic drift term is now modified by the score function $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$. This combined term acts as a powerful systematic force, examining the noisy image and aggressively steering it uphill toward the peaks of realistic data. However, the reverse process still contains a stochastic component, $d\bar{\mathbf{w}}$, which is the reverse Wiener process. Just as in our discrete generation steps, this continuous random jitter is absolutely essential. The reverse Wiener process constantly shakes the image as it flows uphill, preventing it from getting stuck in smooth, blurry, average states and forcing it to resolve into sharp, specific textures.

This continuous perspective unites the concepts we have covered. During the forward process, the added random noise essentially acts as a force pushing our image downhill into the flat valley of pure noise. Because the noise pushes downhill, the score function, which strictly points uphill toward realistic data, points in the exact opposite direction. Mathematically, the continuous score function is directly proportional to the negative of the added noise. Therefore, when we trained our neural network $\epsilon_{\theta}(\mathbf{x}_t, t)$ to isolate the noise hidden in an image, we were actively estimating the score function. Knowing this score function gives us the necessary information to define the exact reverse SDE. By following this continuous reverse equation, we can start with a sample of pure noise and continuously adjust our image based on the score function, which provides a systematic force pushing it uphill. Simultaneously, the reverse Wiener process injects a continuous random thermal jitter,

preventing the generation process from getting stuck and ensuring it thoroughly explores the space of possibilities. This continuous random walk up the probability mountain is the formal mathematical equivalent of the [Langevin dynamics](#) we discussed earlier, providing a rigorous and smooth flow to guide formless noise back into a sharp, realistic image.

9 Deterministic Generation and DDIM

The continuous reverse stochastic differential equation effectively describes how to turn pure noise into structured data, but it relies on taking extremely small, careful steps. In our mental image of the probability landscape, the random thermal jitter keeps the generation process actively exploring. However, if we take a step that is too large, the stochastic randomness might accidentally throw us off a steep cliff instead of safely guiding us up the mountain.

An important result from statistical physics, rooted in what is known as the Fokker-Planck equation, offers an elegant alternative. It states that for every stochastic process that evolves a probability distribution over time, there exists a perfectly deterministic equation that achieves the exact same evolution of the distribution. This deterministic counterpart is called the [Probability Flow ODE](#) (Ordinary Differential Equation):

Probability
Flow ODE

$$d\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \right] dt$$

Comparing this equation with the continuous reverse SDE from the previous section, two differences stand out: we drop the random reverse Wiener process entirely, and we introduce a new factor of one-half to the diffusion term. This factor of one-half is mathematically crucial. In a stochastic process, random noise actively spreads out the probability distribution over time. If we remove that random jitter, we must replace it with a deterministic force that replicates that exact same spreading effect. The mathematical laws of probability tell us that scaling our score function by exactly one-half perfectly mimics the outward push of diffusion, ensuring the distribution of images at any time step remains identically matched to the original stochastic process.

The transition from a stochastic to a deterministic equation changes the fundamental nature of the generation process. In the stochastic setting, the path traces a jagged, erratic trajectory, much like a hiker stumbling up a slope while being constantly shoved by random winds. Because of this randomness, if you start with a specific point of pure noise and run the reverse process twice, you will end up with two completely different generated images.

In contrast, the Probability Flow ODE acts like a smooth, unvarying river flowing uphill. A specific starting point in the flat plains of pure noise is permanently tethered to a unique, specific mountain peak of realistic data by a continuous, mathematically precise curve. This deterministic viewpoint is highly powerful because it bridges diffusion models with another family of generative models called [Normalizing Flows](#), a concept sometimes referred to as flow matching. Because the path is completely deterministic, the mapping between noise and data is perfectly invertible. If you take a generated image and run the ODE forward,

you will arrive at a unique, specific noise vector. Running that exact noise vector backward through the ODE yields the exact same image. This removes the uncertainty from the generation phase and gives us profound control over the relationship between chaotic noise and structured images, a property that becomes highly useful for targeted image editing.

Because the Probability Flow ODE traces such a smooth, gentle curve, we can use numerical shortcuts to take much larger mathematical leaps along the path. We no longer need to evaluate our model at thousands of microscopic time steps to avoid falling off the probability mountain. This insight leads to a highly efficient generation technique known as [Denoising Diffusion Implicit Models](#), commonly referred to as [DDIM](#). In the DDIM framework, we fully embrace the deterministic path and skip large chunks of time, for instance stepping directly from $t = 1000$ to 950, then 900, and so on.

Denoising
Diffusion
Implicit
Models

Math: DDIM Update Step

The deterministic DDIM update to jump from a noisy state $\mathbf{x}_t$ directly to a much cleaner state $\mathbf{x}_{t-s}$ (skipping s steps) is formulated as:

$$\mathbf{x}_{t-s} = \sqrt{\bar{\alpha}_{t-s}} \left(\underbrace{\frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}}_{\text{Predicted clean image } \mathbf{x}_0} \right) + \underbrace{\sqrt{1 - \bar{\alpha}_{t-s}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)}_{\text{Direction pointing to noise}}$$

While this equation might look a bit intimidating, its geometric interpretation is quite intuitive.¹ At any point t , the network examines the current noisy image and makes its best guess of what the final, perfectly clean image $\mathbf{x}_0$ looks like. This prediction is represented by the first term in the equation, scaled by the signal factor of our target step. Then, to land accurately on the intermediate target step $\mathbf{x}_{t-s}$, we add back a specific amount of noise. The second term uses the network's predicted noise $\boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)$ as a directional arrow, scaling it precisely to re-noise the clean prediction.

A natural question arises: if DDIM allows us to take large steps and we have a perfect estimate of the final image, why can we not simply take one massive leap from pure noise all the way to a clean image? The answer lies in the shape of the probability landscape. Even with a perfectly trained model, the vector field represented by our score function forms a curved manifold. The model's prediction at time t only gives us a local tangent, pointing in the correct direction for the immediate vicinity. Taking one massive linear jump along this initial tangent would cause us to vastly overshoot the curve and fall off the probability mountain. Therefore, we still need multiple steps, typically 20 to 50 instead of the original 1000, to carefully navigate the curve and safely arrive at a realistic image.

It is crucial to emphasize that DDIM is exclusively a modification to the sampling procedure during generation. The neural network architecture, the training data, and the underlying [DDPM](#) objective we established to estimate the noise remain exactly the same. We only

¹You might wonder where the factor of one-half from the continuous ODE went in this update step. When we use mathematical techniques to solve the continuous ODE over discrete time intervals, the process of integration transforms the continuous rates into the discrete, geometric scaling factors based on $\bar{\alpha}$. The one-half is naturally absorbed into the derivation of these specific square root fractions, resulting in the clean, discrete formula we use in practice.

train a single model, and once training is complete, we gain the flexibility to choose our path: we can generate images slowly and diversely using stochastic dynamics, or rapidly and invertibly using the deterministic DDIM sampler.

10 Compressing the Problem: Latent Diffusion

While the DDIM sampler helps to significantly reduce the generation time by allowing to skip intermediate generation steps, generating high-resolution imagery still demands staggering computational resources. If we want to generate a crisp image with a height and width of 1024 pixels, our neural network must process and predict millions of individual pixel values at every single step. This massive spatial dimensionality makes training and running pixel-space diffusion models extremely slow and memory-intensive, effectively putting them out of reach for standard consumer hardware. To make diffusion models practical for everyday use, we must find a way to compress the spatial dimensions of the problem itself.

To achieve this, we can draw upon a tool we introduced in the context of unsupervised learning: the [Variational Autoencoder](#) (VAE). A natural image contains a massive amount of redundant, high-frequency spatial information that is largely imperceptible to the human eye. When looking at a photograph, the exact mathematical color value of a single microscopic pixel is rarely important. What truly matters are the broader semantic concepts, like the shape of a dog’s ear, the overarching geometry of a car, or the texture of a brick wall. We can train a VAE entirely separately on our image dataset, with the sole purpose of identifying these concepts to achieve spatial compression and accurate reconstruction. Once this VAE is fully trained, we freeze its parameters so they no longer change.

We then take our entire training dataset of images and use the pre-trained [encoder](#) network E to squash the high-dimensional pixel images into much smaller, information-dense representations. We call this compressed representation a [latent vector](#), which we will denote as $\mathbf{z}_0 = E(\mathbf{x}_0)$. This architectural leap gives rise to [Latent Diffusion Models](#) (LDMs), as visualized in Figure 5.

Latent
Diffusion
Models

The core insight of an LDM is that we do not need to add random noise to raw pixels. Instead, we perform the entire forward and reverse diffusion Markov chain exclusively within this compressed [latent space](#). Because the latent space is drastically smaller than the pixel space, often compressing spatial dimensions by a factor of eight or more, the diffusion process runs noticeably faster and requires only a fraction of the (video) memory. We mathematically add noise to the clean latent vector to create a noisy latent state $\mathbf{z}_t$, and we train our smaller, faster U-Net to predict the noise within this abstract concept space. The mathematical objective remains functionally identical to the one we established for standard diffusion, but it is now calculated entirely using latent vectors:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{z}_0, \epsilon, t} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} \mathbf{z}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

During the generation phase, we simply start with a small grid of pure random noise in the latent space and iteratively denoise it using our trained U-Net, comfortably utilizing

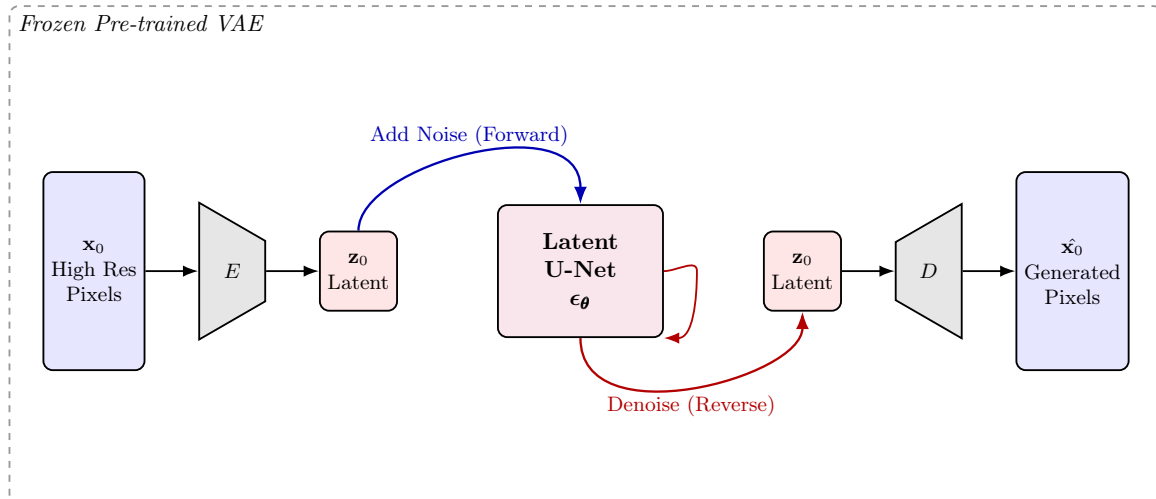


Figure 5: The architecture of Latent Diffusion Models. By moving the heavy iterative denoising process out of the high-resolution pixel space and into the compact, compressed latent space provided by an autoencoder, the model runs significantly faster and requires far less memory.

the rapid DDIM sampler to skip steps if we choose. Once the network has successfully generated a clean, novel latent vector $\mathbf{z}_0$, we pass this vector through the frozen [decoder](#) network D . The decoder unpacks the abstract, compressed concepts back into millions of crisp pixels, yielding our final generated, high-resolution image $\hat{\mathbf{x}}_0 = D(\mathbf{z}_0)$. By decoupling the conceptual generation process, which happens efficiently in the tiny latent space, from the high-frequency pixel rendering, which is handled entirely by the VAE decoder, Latent Diffusion can be smoothly run on ordinary consumer laptops without compromising on visual quality.

11 Excursus: Bridging Modalities with Contrastive Learning

Up to this point, our mathematical framework only describes an unconditional generative model: it turns pure noise into random images from the training distribution. However, an unconditional generator is of limited practical use. We typically want to steer the model using a human instruction, such as asking it to draw “a cat drinking a coffee.” Here we face a fundamental [multimodal](#) problem. Our neural network only understands the mathematical language of spatial tensors, while our prompt is written in the semantic language of human text.

Before we can teach our diffusion model to follow instructions, we must first find a way to bridge this gap. We need a shared mathematical environment—which we will call the [joint embedding space](#) $\mathcal{E}$ —where visual and textual concepts naturally align, conceptually depicted in Figure 6. This challenge is highly relevant to cognitive science, as it mirrors the process of grounding language: understanding how the linguistic representation of a concept maps directly to its perceptual features in the real world. To achieve this in deep learning, we take a

joint
embedding
space

brief excursus to explore a pre-trained framework called [CLIP](#) (Contrastive Language-Image Pre-training).

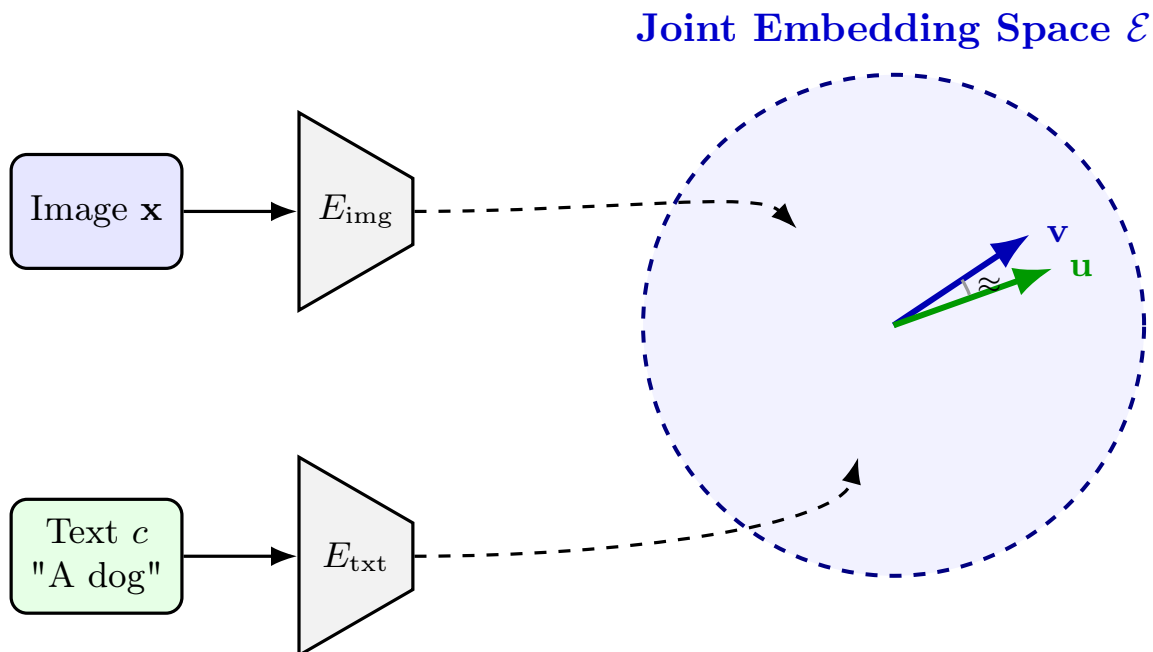


Figure 6: The CLIP joint embedding space. The model trains two separate encoders—one for images and one for text—to map data into a single, shared mathematical landscape. In this space, the vector for the word “dog” naturally points in the same direction as the visual features extracted from a picture of a dog.

The goal of CLIP is to construct this joint embedding space $\mathcal{E}$ using two separate neural network [encoders](#): an image encoder E_{img} and a text encoder E_{txt} . The image encoder acts as a mathematical function mapping an image from the raw pixel space $\mathcal{X}$ into the joint space, written as $E_{\text{img}} : \mathcal{X} \rightarrow \mathcal{E}$. Similarly, the text encoder maps a string of words from the text space $\mathcal{T}$ into the exact same joint space, written as $E_{\text{txt}} : \mathcal{T} \rightarrow \mathcal{E}$.

During its independent training phase, CLIP is presented with a large dataset of images paired with their descriptive text captions. Instead of trying to perform a highly complex task like predicting the exact words of the caption from the image pixel by pixel, it relies on a [contrastive learning](#) objective. The core idea is simple: teach the model to correctly identify which image goes with which text out of a random lineup.

[contrastive learning](#)

To formalize this, let us take a [mini-batch](#) of m paired images and captions. We pass all the images through the image encoder to obtain a set of compressed image vectors $v_i = E_{\text{img}}(x_i) \in \mathcal{E}$, and all the captions through the text encoder to obtain text vectors $u_i = E_{\text{txt}}(c_i) \in \mathcal{E}$. Because both encoders are designed to output vectors of the exact same dimensionality within the shared space $\mathcal{E}$, we can measure the semantic alignment between any given image vector v_i and any text vector u_j by calculating their [scalar product](#). We store these products in a similarity matrix $\mathbf{S}$:

$$S_{i,j} = v_i \cdot u_j$$

If an image and a text share the same meaning, their vectors should point in the same direction within $\mathcal{E}$, resulting in a large positive scalar product. If they are unrelated, the product should be small or negative. In our batch of m pairs, there are exactly m correct matching pairs (where $i = j$, forming the diagonal of our matrix) and $m^2 - m$ incorrect combinations (where $i \neq j$, forming all the off-diagonal entries).

We can transform this matrix into a concrete loss function by treating it as a classification problem. For any specific image i , the network must classify which of the m available captions is the correct one. We can take the i -th row of our similarity matrix and apply the [softmax](#) function, which we established in the context of multiclass classification, to convert the scalar products into a probability distribution:

$$\hat{p}_{i,j} = \frac{\exp(S_{i,j}/\tau)}{\sum_{k=1}^m \exp(S_{i,k}/\tau)}$$

Here, τ is a temperature scaling parameter, a concept we encountered earlier, which helps to sharpen or soften the probability distribution. Once we have these probabilities, we can apply the standard [cross-entropy](#) loss. We want to maximize the probability of the correct pair $\hat{p}_{i,i}$, which is equivalent to minimizing its negative logarithm:

$$\mathcal{L}_{\text{image-to-text}} = -\frac{1}{m} \sum_{i=1}^m \log \frac{\exp(S_{i,i}/\tau)}{\sum_{k=1}^m \exp(S_{i,k}/\tau)}$$

To ensure the space is fully symmetric, CLIP simultaneously calculates this loss in the other direction—asking the text to find the correct image among the columns of the matrix—and averages the two results to form the final $\mathcal{L}_{\text{contrastive}}$.

While this mathematical formulation is clear, making it work in practice introduces a significant challenge: the problem of [hard negatives](#). If we use a standard batch size of 32, a picture of a dog will likely be compared against captions describing a car, an apple, or a building. The network can easily solve this matching game by looking at superficial features like background color, without ever truly understanding the objects. To force the network to learn deep, nuanced semantic representations, it needs to be challenged with subtle differences, such as distinguishing “a black dog running” from “a black dog sitting.” In contrastive learning, this is typically solved by using massive batch sizes. By processing tens of thousands of image-text pairs in a single batch, the probability of naturally including similar but subtly different examples increases, forcing the model to deeply understand the language to minimize the loss.

[hard
negatives](#)

Through this systematic task of contrasting correct pairings against incorrect ones across diverse data, the encoders learn a visually grounded understanding of language. In the joint embedding space $\mathcal{E}$, the mathematical vector for the word “dog” ends up pointing in the exact same direction as the visual features extracted from a photograph of a dog. By successfully mapping human language and raw pixels into a shared conceptual landscape,

contrastive learning provides us with the exact foundational tool we need to eventually teach our diffusion models to comprehend and follow our written instructions.

12 Guiding the Generation: Conditioning and Classifier-Free Guidance

For our immediate goal of generating specific images, the joint embedding space $\mathcal{E}$ provides the exact tool we were missing. We can extract the fully trained, frozen text encoder E_{txt} and use it to translate any human instruction c from the text space $\mathcal{T}$ into a vectorial representation in the joint space. This output is our **conditioning vector**, denoted as $\mathbf{c} = E_{\text{txt}}(c)$, where $\mathbf{c} \in \mathcal{E}$. Our next challenge is to integrate this semantic vector into the generation process. We need to upgrade our U-Net from a model that simply predicts generic noise $\epsilon_{\theta}(\mathbf{z}_t, t)$ into a **conditional model** $\epsilon_{\theta}(\mathbf{z}_t, t, \mathbf{c})$ that estimates noise based on the specific instructions provided by the user.

conditioning
vector

We cannot simply add the text vector $\mathbf{c}$ to the image pixels or latents. The U-Net processes spatial grids—where information is organized by width, height, and channels—while the conditioning vector is a dense, abstract array of semantic features. To merge these completely different formats, modern U-Nets incorporate a mechanism from natural language processing called **cross-attention**. While we will explore the precise mathematical mechanics of attention in a later chapter, we can intuitively think of cross-attention as a flexible, internal search engine embedded within the layers of the U-Net.

cross-
attention

As the U-Net processes the noisy image $\mathbf{z}_t$, the intermediate spatial feature maps act as “queries”. They actively scan the conditioning vector $\mathbf{c} \in \mathcal{E}$ to look for relevant information. For example, if a region of the noisy image vaguely resembles an animal’s face, those specific spatial locations will mathematically pay attention to the part of the text vector that encodes the word “cat”. The text vector returns guidance, updating the image features and directing the denoising process to specifically sculpt a cat’s features out of the static in that exact location. By injecting these cross-attention modules at multiple resolutions throughout both the **encoder** and **decoder** pathways of the U-Net, the model ensures that the global composition and the finest details are all strictly governed by the semantic meaning of the text prompt.

To teach the U-Net how to use this new text information, we must adjust our training procedure. During the training loop, whenever we randomly sample a clean image and corrupt it to a specific time step t , we also pass its corresponding text caption through the frozen text encoder to generate $\mathbf{c}$. We then provide this continuous vector to the U-Net alongside the noisy image. The overall training objective remains the same **squared error loss**, but the network now learns to minimize the error by actively relying on the text description.

However, if we strictly train the network this way, it introduces a subtle problem. Neural networks tend to be conservative and often rely on the easiest path to minimize their loss. If we give a purely conditionally trained model a complex, highly specific prompt, it might still generate a safe, generic image that loosely resembles the prompt but ignores the finer

details, because generic images make up the vast majority of its training data. The prompt effectively acts as a gentle suggestion rather than a strict command.

To solve this, we want to mathematically force the model to aggressively follow our instructions. We achieve this through a technique called **Classifier-Free Guidance** (CFG). For CFG to work, the U-Net must know how to generate images both conditionally (guided by specific text) and unconditionally (guided only by the general dataset distribution). Therefore, we make a small but crucial change to our training loop: roughly ten to twenty percent of the time, we intentionally drop the text caption. In its place, we supply the network with an unconditional embedding vector derived from an empty **null prompt** $\emptyset$. For consistency in our notation, we will use $\emptyset$ to indicate that the network is being conditioned on this empty representation in $\mathcal{E}$. This forces the same U-Net to learn two distinct behaviors simultaneously.

Classifier-Free
Guidance

null prompt

On the one hand, when the text prompt is present, the network learns the conditional behavior. In terms of our probability landscape, it learns to find the steepest path toward the specific, narrow peaks corresponding to the text description. On the other hand, when the null prompt is given, the network learns the unconditional behavior, finding the general path toward any valid image regardless of its specific content.

During the generation phase, we take full advantage of this dual capability. At every single time step t , we do not run the U-Net just once; we run it twice. First, we feed the U-Net our noisy state and the actual text embedding $\mathbf{c}$, obtaining a conditional noise prediction. Second, we feed the U-Net the exact same noisy state, but we replace the text embedding with the unconditional embedding of the null prompt $\emptyset$, obtaining an unconditional noise prediction.

We now have two vectors: what the model thinks the noise is when it is looking at the text, and what it thinks the noise is when it is completely ignoring the text. The mathematical difference between these two predictions isolates the visual elements that are specific to our text prompt. To force the model to listen to us, we take the unconditional prediction and add that specific difference back into it, multiplied by a scalar value called the **guidance scale** γ . This dynamic is illustrated in Figure 7.

guidance
scale

If we set the guidance scale $\gamma = 1$, the unconditional terms cancel out, and we simply get the standard conditional prediction. However, if we set γ to a higher value, commonly between 4 and 7, we artificially amplify the effect of the text prompt. Recalling our continuous flows, this operation actively alters the vector field. It steepens the gradient of the probability landscape, pushing the generation path away from the broad valleys of generic images and aggressively steering it toward the specific peaks defined by the text prompt.

13 Editing Reality: Deterministic Inversion

We have seen how diffusion models can conjure entirely new images from scratch, but their capabilities also extend to manipulating our existing reality. Imagine you have a photograph of a plain house, and you want to use a diffusion model to cover the roof in snow or change its architectural style. If we simply add random noise to the photo and ask the model to

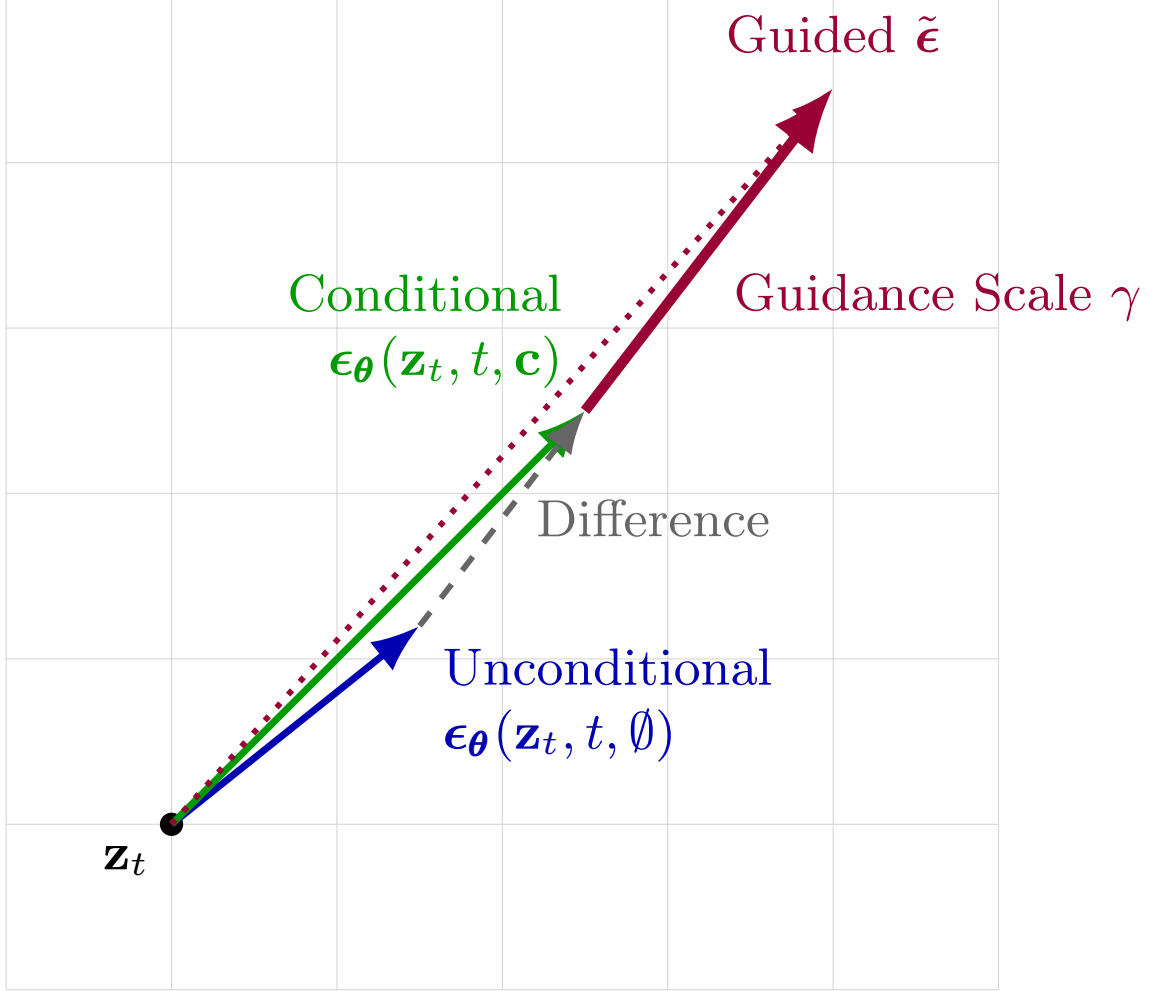


Figure 7: Classifier-Free Guidance (CFG). By calculating the difference between the conditional prediction (guided by text) and the unconditional prediction (generic generation), the model isolates the specific features corresponding to the prompt. Multiplying this difference by a guidance scale amplifies the effect, forcing the model to closely adhere to the instructions.

generate a snowy house, the random thermal jittering of the standard generation process will destroy the spatial layout. The model will likely generate a completely different house, losing the specific identity of the original photograph.

To edit an image while preserving its underlying structure, we can take advantage of the deterministic path of the Probability Flow ODE and the DDIM sampler. Because this deterministic path acts like an unvarying river, it is mathematically invertible. This technique is known as [deterministic inversion](#). We begin by compressing our real photograph $\mathbf{x}_0$ into a latent vector $\mathbf{z}_0$ and providing a text vector in the joint space $\mathcal{E}$ that accurately describes the original image, which we will denote as $\mathbf{c}$. Instead of starting with noise, we start with our image and use the deterministic equations in reverse to carefully trace the path outward into pure chaos. This allows us to find the exact sequence of latent states $\mathbf{z}_t$ and the corresponding specific noise predictions—which we will call our target noise ϵ_t^* —that uniquely map our original photograph to a vector of pure noise.

[deterministic inversion](#)

Once we have mapped our photograph into the model’s noise space, we can initiate the editing process. We keep that specific starting noise footprint, but we swap our original text conditioning $\mathbf{c}$ for a new embedding $\mathbf{c}'$ that represents our editing prompt, such as “a snowy house”. We then run the standard deterministic generation process forward, moving from noise back to data. Because we start from the exact noise vector of the original image, the deterministic path ensures that the broad spatial layout and the geometry remain largely intact. The cross-attention layers, now guided by the new prompt $\mathbf{c}'$, will only alter the specific features necessary to satisfy the new text, laying snow on the roof while leaving the shape of the house identical.

While this mathematical reversibility works smoothly in theory, applying it in practice introduces a significant hurdle. To ensure the model strongly follows our editing prompt $\mathbf{c}'$, we must use [Classifier-Free Guidance](#) (CFG). However, as we established earlier, CFG fundamentally alters the underlying vector field. By artificially amplifying the estimated score function to push the image toward text-specific features, CFG actively changes the rules of the environment. This alteration breaks the mathematical reversibility of the deterministic ODE path. If we use CFG during the generation pass, the trajectory will deviate from the unguided path we carefully calculated during the inversion pass. As a result of this divergence, even the parts of the image we did not want to change, like the background or the lighting, will shift unrecognizably.

To resolve this tension, we rely on a technique called [Null-text Inversion](#). The core idea is to carefully preserve the original trajectory of the image without turning off the powerful steering effect of CFG. We can achieve this by framing the problem as a small, targeted optimization task, reusing the concept of [gradient descent](#) we established in our very first chapters. However, there is a fascinating twist: instead of updating the parameters θ of our neural network—which remain completely frozen—we will use backpropagation to systematically adjust the continuous values of the unconditional embedding vector (the null prompt) itself.

[Null-text Inversion](#)

For every time step t , our goal is to find an optimized unconditional embedding vector $\emptyset_t \in \mathcal{E}$ that acts as a mathematical counterweight. We want to adjust this empty prompt so that when we apply the CFG formula using our *original* text description $\mathbf{c}$, the resulting guided

noise prediction perfectly matches the exact target noise ϵ_t^* we recorded during our initial inversion. We can formalize this with a straightforward [squared error loss](#):

$$\mathcal{L}(\emptyset_t) = \left\| \epsilon_t^* - \left(\epsilon_{\theta}(\mathbf{z}_t, t, \emptyset_t) + \gamma \cdot (\epsilon_{\theta}(\mathbf{z}_t, t, \mathbf{c}) - \epsilon_{\theta}(\mathbf{z}_t, t, \emptyset_t)) \right) \right\|^2$$

By repeatedly calculating this loss and taking small gradient descent steps to update $\emptyset_t$, we effectively absorb the mathematical deviation caused by CFG entirely into these new, time-specific unconditional embeddings. We repeat this quick optimization for every step, yielding a complete sequence of optimized null prompts.

Finally, we perform the actual editing pass. We start from the pure noise of the inverted image and generate forward, but now we apply CFG using our *new* editing prompt $\mathbf{c}'$ alongside our freshly optimized null embeddings $\emptyset_t$:

Math: Null-text Inverted CFG Generation

$$\tilde{\epsilon} = \epsilon_{\theta}(\mathbf{z}_t, t, \emptyset_t) + \gamma \cdot (\epsilon_{\theta}(\mathbf{z}_t, t, \mathbf{c}') - \epsilon_{\theta}(\mathbf{z}_t, t, \emptyset_t))$$

The conditional part of the equation actively pushes the image toward the new semantic features of the edit, but the unconditional part acts as a customized anchor to the original reality. This technique corrects the trajectory drift, allowing us to alter specific details of a photograph while leaving everything else intact. It demonstrates that diffusion models do not just memorize static pixels, but rather learn a continuous, structurally logical, and highly manipulable space of semantic concepts.

14 Straightening the Path: Flow Matching and Optimal Transport

In our exploration of deterministic generation and the DDIM sampler, we visualized image generation as a smooth river flowing uphill. We also briefly noted that this deterministic perspective connects diffusion to a concept called [flow matching](#). However, because the continuous paths of standard diffusion models are mathematically derived from the physical metaphor of thermal dynamics, they remain highly curved. Imagine navigating a winding mountain road; even if the surface is perfectly smooth, you must take small, careful steps to avoid veering off the edge. This curvature forces the U-Net to evaluate the image dozens of times to stay on track. Recent research asks a fundamental geometric question: if we no longer need to simulate physical diffusion and are simply using deterministic paths to connect data and noise, why not make the paths of our flows perfectly straight?

This conceptual shift is the core of modern flow matching techniques. Instead of predicting the noise, we reframe time as a continuous variable t moving linearly between 0 (structured data) and 1 (pure noise). The mathematical goal is to define a perfectly straight line connecting a clean image $\mathbf{x}_0$ directly to a sample of pure noise ϵ . We can formulate this straight path simply as:

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\epsilon$$

Because the path is a straight line, the rate of change, or the velocity $\mathbf{v}$, is constant. By calculating the derivative of this equation with respect to time, the true velocity is simply the difference between the endpoints: $\mathbf{v} = \epsilon - \mathbf{x}_0$. We then train a neural network $\mathbf{v}_\theta(\mathbf{x}_t, t)$ to predict this directional vector, minimizing the [squared error loss](#) between the true velocity and the predicted velocity.

While a straight line is the shortest distance between two points, a critical issue arises when we look at the entire probability landscape. If we randomly pair any noise sample with any training image, the straight lines connecting them will inevitably crisscross. If the neural network lands on an intersection point where multiple paths cross, it will receive conflicting velocity targets. It will not know which specific path to follow, leading to blurry, averaged, or chaotic outputs. To make straight paths mathematically viable, we must systematically organize the traffic.

This perspective leads to a mathematical framework called [Optimal Transport](#). We can understand this through the classic Earth Mover’s metaphor. Imagine the distribution of training images as a complex mountain of dirt, and the distribution of noise as a vast, flat plain. Optimal Transport asks for the cheapest, most energy-efficient way to move the dirt to form the plain. By “cheapest”, the mathematics specifically means minimizing the total distance traveled by all particles combined. Applied to flow matching, Optimal Transport carefully pairs each noise vector with the closest possible target image before drawing the lines. By actively organizing these pairings during training, it ensures the straight paths are parallel and non-intersecting, completely eliminating the navigational confusion for the neural network, as compared in Figure 8.

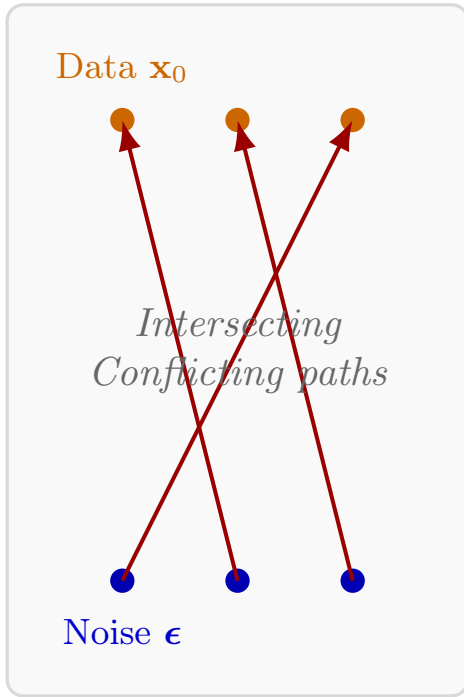
[Optimal
Transport](#)

We can also extend this geometric optimization back to the probabilistic, stochastic paths using a concept known as [Schrödinger Bridges](#). Formulated by the physicist Erwin Schrödinger in the 1930s, the problem asks: if we observe a cloud of particles at an initial state, and later observe them at a final state, what is the most likely set of trajectories they took, assuming they were subject to random thermal noise? While Optimal Transport finds the absolute shortest deterministic paths, Schrödinger Bridges find the most efficient paths that still incorporate a specific amount of random jitter. By applying this mathematics to diffusion, we can design stochastic processes that destroy information in the most geometrically direct way possible, while still benefiting from the thermal jittering of Langevin dynamics. This allows the model to actively explore the space and ensure sharp, high-frequency details in the final image, but without wasting time wandering along unnecessarily curved routes.

[Schrödinger
Bridges](#)

These advanced geometric techniques represent the cutting edge of current generative research and serve as the mathematical foundation for some of the newest image generation models, such as Stable Diffusion 3 and Flux. By straightening the trajectories and meticulously organizing the mapping between noise and data, flow matching and Optimal Transport allow neural networks to take substantially larger, more reliable steps along the probability landscape. This geometric refinement reduces the required number of generation steps from dozens down to as few as one to four, considerably cutting computational costs and paving the way for highly efficient visual generation.

Standard Matching



Optimal Transport

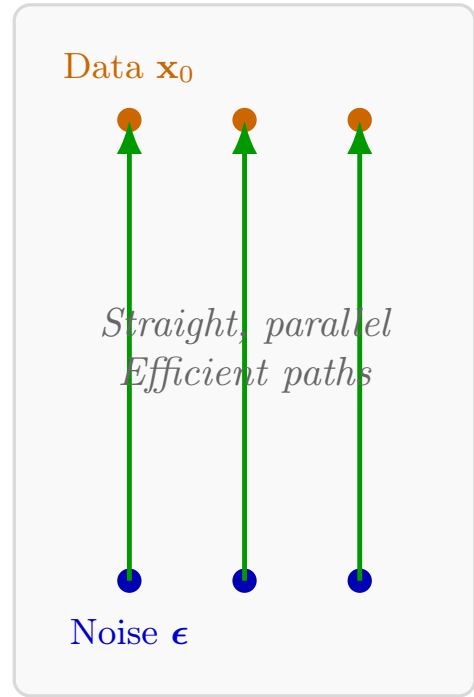


Figure 8: Optimal Transport and Flow Matching. On the left, randomly matching noise to data creates intersecting, confusing paths for the neural network. On the right, Optimal Transport mathematically pairs noise to the closest data point, creating clean, parallel, straight trajectories. This allows the model to generate images in far fewer steps.

15 Summary and Outlook

In this chapter, we explored the world of diffusion models, a highly effective paradigm for creating generative models. We started with the physical intuition of thermodynamics, comparing the natural increase of entropy to the systematic destruction of an image using Gaussian noise. By framing this destruction as a Markov process, we trained a U-Net to perform [score matching](#), effectively learning to predict and subtract noise to reverse the flow of time and guide a chaotic state back to a structured image.

We discovered that this discrete stochastic process can be expanded into a continuous flow using Stochastic Differential Equations. This continuous framework revealed that the random diffusion process has a strictly deterministic counterpart in the Probability Flow ODE. This deterministic perspective, embodied in the DDIM sampler, enabled fast, reversible mappings between noise and data, establishing a conceptual bridge to [Normalizing Flows](#).

To make these computationally demanding models practical for everyday hardware, we examined [Latent Diffusion Models](#). These architectures drastically reduce memory and processing requirements by performing the entire diffusion process within the compressed conceptual space of a pre-trained autoencoder. We then tackled the challenge of multimodal generation by introducing the [CLIP](#) model as a crucial mathematical bridge between textual instructions and visual features. We explored how [Classifier-Free Guidance](#) artificially steepens the probability landscape to force the network to strictly adhere to these textual prompts, and how techniques like [Null-text Inversion](#) leverage the deterministic ODE to allow for the precise, semantic editing of real-world photographs.

Finally, we looked at the geometric future of generative research. We noted how frameworks like [Flow Matching](#) and [Optimal Transport](#) help to systematically untangle and straighten the generative trajectories, making the journey from noise to data remarkably short, efficient, and reliable.

References and further reading

The conceptual foundation of using non-equilibrium thermodynamics to build generative models was first introduced by Sohl-Dickstein et al. (Sohl-Dickstein et al. 2015). This physical inspiration was later refined and popularized within the deep learning community by Ho, Jain, and Abbeel (Ho et al. 2020), who formulated the [Denoising Diffusion Probabilistic Model](#) (DDPM) and established the practical mathematical framework of predicting noise.

The continuous perspective, uniting discrete diffusion steps and score-based models through the lens of Stochastic Differential Equations, was elegantly detailed by Song et al. (Song et al. 2021). This paper formalized the Probability Flow ODE, which serves as the theoretical basis for deterministic sampling methods and connects diffusion back to the broader family of continuous generative flows.

To address the immense computational costs of pixel-space diffusion, Rombach et al. (Rombach et al. 2022) proposed [Latent Diffusion Models](#). By shifting the iterative denoising

process into the compressed space of a pre-trained autoencoder, their work laid the groundwork for highly efficient, high-resolution models, most notably Stable Diffusion.

The crucial multimodal bridge that allows these generative models to follow text instructions relies heavily on [Contrastive Language-Image Pre-training](#) (CLIP), introduced by Radford et al. (Radford et al. 2021). Their research demonstrates how large-scale contrastive learning can create a robust joint embedding space that securely connects human language to visual features.

For a highly accessible, visually driven explanation of these advanced concepts, Prince’s textbook *Understanding Deep Learning* (Prince 2023) provides excellent, deeply illustrated chapters on both diffusion models and flow matching. The text makes the complex probability landscapes, ODE formulations, and optimal transport geometric mappings highly intuitive.

Ho, Jonathan, Ajay Jain, and Pieter Abbeel. 2020. “Denoising Diffusion Probabilistic Models.” *Advances in Neural Information Processing Systems (NeurIPS)* 33: 6840–51.

Prince, Simon J. D. 2023. *Understanding Deep Learning*. The MIT Press. <http://udlbook.com>.

Radford, Alec, Jong Wook Kim, Chris Hallacy, et al. 2021. “Learning Transferable Visual Models from Natural Language Supervision.” *Proceedings of the 38th International Conference on Machine Learning (ICML)* 139: 8748–63.

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. “High-Resolution Image Synthesis with Latent Diffusion Models.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10684–95.

Sohl-Dickstein, Jascha, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. “Deep Unsupervised Learning Using Nonequilibrium Thermodynamics.” *Proceedings of the 32nd International Conference on Machine Learning (ICML)* 37: 2256–65.

Song, Yang, Jascha Sohl-Dickstein, Diederik P Kingma, Arvind Kumar, Stefano Ermon, and Ben Poole. 2021. “Score-Based Generative Modeling Through Stochastic Differential Equations.” *International Conference on Learning Representations (ICLR)*.